CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment 2

Name	: Johnson Kumar	**UID**	: 23BCS12654
Branch	: CSE	**Section**	: KRG-1A
Semester	: Sixth	**Date of exp.**	: 11/01/26
Subject	: System Design	**Subject Code**	: 23CSH-314

AIM

We have to design an Online shopping platform similar to Amazon / Flipkart that will allow users to purchase mobiles, laptops, cameras, clothes etc.

OBJECTIVE:

- To understand functional and non-functional requirements of a large-scale e-commerce system.
- To design RESTful APIs for product browsing, cart management, order placement, and payment processing.
- To identify core entities such as User, Product, Cart, Order, Payment, and Inventory.
- To design high-level and low-level architecture for a scalable, fault-tolerant shopping platform.
- To evaluate database choices (SQL vs NoSQL) and apply consistency models for inventory and order management.
- To implement secure authentication, authorization, and payment gateway integration ensuring reliability and user trust.

TOOLS USED

- **Python** – Backend logic implementation and URL generation algorithms.
- **Flask** – Lightweight web framework for developing RESTful APIs.
- **Draw.io** – Designing system architecture diagrams (HLD & LLD).

SYSTEM REQUIREMENTS:

A. Functional Requirements

- User should be able to search and find the products based on product title or names.
- User should be able to view the details of the product llike description, image, available quantity, review
- User should be able to select the quantity and move the product/item into the cart.
- User should be able to make the payment and should be able to perform the check out.
- User should be able to check the status of the order.
- System should be able to manage purchase of items having limited stocks.

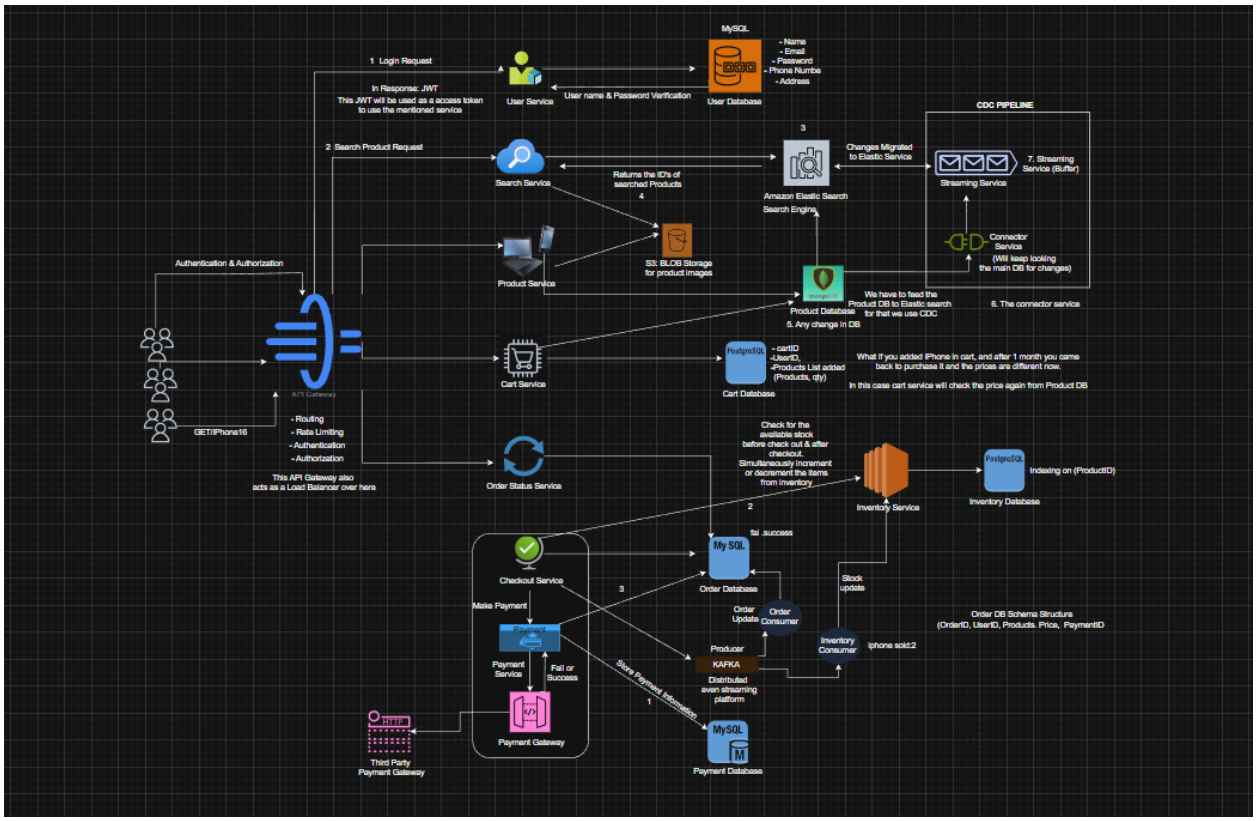
B. Non-Functional Requirements

- Target Scale: 100Million DAU with 10 orders processed per second.
- Consistency & Availability: Here for this system we need both as per the Target Scale.
    - Availability: As per the FR, we know that client should be able to search for the products efficiently means we need:
      >High Availability(on Products search)
    - Consistency: The system should be highly consistence for our two important modules:
      >Payment and the Order Placing.
      >Inventory Management
- Latency: Required: ~200 ms
- Scaling: Horizontal / Vertical

HIGH LEVEL DESIGN (HLD):

Core-Entites of the System:

1. User / Client
2. Products
3. Cart
4. Orders
5. Checkout followed by Payment



LOW LEVEL DESIGN (LLD):

API Designing:

1. GET API Call: Prod_Search

   Https://Local_Host/products/search_item = {Search_keywords}
   HTTP Req
   GET: <iPhone 16>
   HTTP Res
   List<ProductID:iPhone>

2. GET API Call: View Product Details

   Https://Local_Host/products/{product_id}
   HTTP Req
   GET: <Product id = 17>
   HTTP Res
   Product_id:17,
   Name: iPhone 17,

Color: Navy Blue,
Price: $1099,
Image Thumbnail: URL_Image

3. POST API Call: Item add in cart

Https://Local_Host/cart/add_products
HTTP Req
Product_id = 17,
Product id = 16
HTTP Req Header
User id: 04
HTTP Res
Cart id: 101

4. PUT API Call: To update any order in the cart

Let's Suppose you want to add one more product into the cart.

5. DELETE API Call: To remove any item from the cart

Let's Suppose you want to delete one more product into the cart.

6. POST API Call: for check out & Payement

Https://Local_Host/checkout -> {post body}
HTTP REQ
All products ID's,
Total Quantity,
Total Price
HTTP RES
Order ID
Https://Local_Host/payment -> {post body}
HTTP REQ
Order ID,
Payment Type,
Payment Mode
HTTP RES
Confirmation Status: Succes / Fail

7. GET API Call: Order Status

Https://Local_Host/orde_status = {order_id}

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

LEARNING OUTCOMES:

- We learned to analyze functional and non-functional requirements of a large-scale e-commerce system.
- We got to know how to design RESTful APIs for product browsing, cart management, and order placement.
- We learned to identify and model core entities such as User, Product, Cart, Order, Payment, and Inventory.
- We got to know the importance of CAP theorem trade-offs and consistency in inventory/order management.
- We learned to design scalable architectures (high-level and low-level) for reliability and fault tolerance.